



PRACTICAL - 3

Implementation of Max-Heap Sort Algorithm

Aim of the Practical:

To implement Heap Sort using a max heap and analyze its running time step by step.

Algorithm:

1. Build a max heap from the input array.
2. Swap the maximum element at the root with the last element.
3. Reduce heap size by one.
4. Heapify the root.
5. Repeat until the heap becomes empty.

Program:

```
#include <iostream>
using namespace std;
void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if (left < n && a[left] > a[largest]) largest = left;
    if (right < n && a[right] > a[largest]) largest = right;
    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}
int main() {
    int a[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(a) / sizeof(a[0]);
    heapSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

5 6 7 11 12 13

Time Analysis:

Derivation of Time Complexity:

Heap Sort has two main phases: build the max heap and repeatedly extract the maximum.

Phase 1 - Build Heap: heapify is called on internal nodes from $n/2 - 1$ down to 0.

The tighter analysis of build-heap sums the work by node height:

$$T_{\text{build}}(n) = \sum_{h=0}^{\log n} [\text{number of nodes of height } h] \cdot O(h)$$

$$T_{\text{build}}(n) = O(n \cdot \sum_{h=0}^{\log n} h / 2^{(h+1)})$$

The infinite series $\sum h/2^{(h+1)}$ converges to a constant, therefore:

$$T_{\text{build}}(n) = O(n)$$

Phase 2 - Repeated extraction: there are $(n - 1)$ extractions.

Each heapify after extraction takes at most $O(\log n)$.

$$T_{\text{extract}}(n) = (n - 1) \cdot O(\log n) = O(n \log n)$$

Total:

$$T(n) = O(n) + O(n \log n) = O(n \log n)$$

Final Time Analysis:

- Best Case: $\Theta(n \log n)$.
- Average Case: $\Theta(n \log n)$.
- Worst Case: $\Theta(n \log n)$.
- Auxiliary Space: $O(\log n)$ with recursive heapify.

Result:

Max-Heap Sort was implemented successfully and its total complexity was obtained by analyzing build-heap and extraction separately.



PRACTICAL - 4

Implementation and Time Analysis of Factorial using Iterative and Recursive Methods

Aim of the Practical:

To implement factorial iteratively and recursively and compare their time and space complexities.

Program:

```
#include <iostream>
using namespace std;
long long factorialIterative(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
    return fact;
}
long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n = 5;
    cout << "Iterative: " << factorialIterative(n) << endl;
    cout << "Recursive: " << factorialRecursive(n) << endl;
}
```

Output:

Iterative: 120
Recursive: 120

Time Analysis:

Iterative Method - Derivation:

The loop starts at 2 and runs through n.

Number of iterations = $n - 1$

Each iteration performs constant work.

$T_{iter}(n) = c(n - 1) + c_0 = \Theta(n)$

Only a fixed number of variables are used, so auxiliary space is $\Theta(1)$.

Recursive method recurrence:

$T_{rec}(n) = T_{rec}(n - 1) + c$

Unrolling:

$$\begin{aligned} T_{rec}(n) &= T_{rec}(n - 2) + 2c \\ &= T_{rec}(1) + (n - 1)c \\ &= \Theta(n) \end{aligned}$$

There are n recursive stack frames in the worst case, so auxiliary space is $\Theta(n)$.

Final Time Analysis:

- Iterative Time: $\Theta(n)$, Space: $\Theta(1)$.
- Recursive Time: $\Theta(n)$, Space: $\Theta(n)$.

Result:

Both methods compute factorial correctly. Their time is linear, but recursion uses linear stack space.